

geosophon工具转mcp

1. <http://192.168.110.138:30080/ecologicalRemoteSensing/mcp>
2. 添加工具

```
def main():
    # 创建服务器
    server = Server("minimal-http-mcp")

    # 定义工具
    @server.list_tools()
    async def list_tools() -> List[Dict[str, Any]]:
        return [
            {
                "name": "标注图片",
                "description": "标注图片",
                "inputSchema": {
                    "type": "object",
                    "properties": {
                        "image_data": {"type": "array", "description": "图片下载地址数组"},
                        "box_points": {"type": "string", "description": "json字符串"},
                        "api_key": {"type": "string", "description": "api_key"},
                        "chat_id": {"type": "string", "description": "chat_id"},
                        "agent_id": {"type": "string", "description": "agent_id"},
                        "model": {"type": "string", "description": "model"},
                    },
                    "required": ["image_data", "box_points", "api_key", "chat_id", "agent_id"]
                }
            }
        ]

    @server.call_tool()
    async def call_tool(name: str, arguments: Dict[str, Any]) -> List[Dict[str, Any]]:
        if name == "标注图片":
            image_data = arguments.get("image_data", [])
            box_points = arguments.get("box_points", "")
            api_key = arguments.get("api_key", "")
            chat_id = arguments.get("chat_id", "")
```

```
"""
    最小化 HTTP MCP 服务器
    只包含最基本的功能
"""
import asyncio
import logging
import uvicorn
from typing import Any, Dict, List
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from mcp.server import Server
from tag_images import start_tag

# 配置日志
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

1 usage
async def main():
    """最小化 HTTP MCP 服务器"""

    # 创建服务器
    server = Server("minimal-http-mcp")

    # 定义工具
    @server.list_tools()
    async def list_tools() -> List[Dict[str, Any]]:
        return [
            {
                "name": "标注图片",
                "description": "标注图片",
                "inputSchema": {
                    "type": "object",
                    "properties": {
                        "image_data": {"type": "array", "description": "图片下载地址数组"},
                        "box_points": {"type": "string", "description": "json字符串"},
                        "api_key": {"type": "string", "description": "api_key"},
                        "chat_id": {"type": "string", "description": "chat_id"},
                        "agent_id": {"type": "string", "description": "agent_id"},
                        "model": {"type": "string", "description": "model"},
                    },
                    "required": ["image_data", "box_points", "api_key", "chat_id", "agent_id"]
                }
            }
        ]

    @server.call_tool()
    async def call_tool(name: str, arguments: Dict[str, Any]) -> List[Dict[str, Any]]:
        if name == "标注图片":
            image_data = arguments.get("image_data", [])
            box_points = arguments.get("box_points", "")
            api_key = arguments.get("api_key", "")
            chat_id = arguments.get("chat_id", "")
```

```
server.py x weather_query.py requirements.txt
25 async def main():
121     15871314160 *
122     @server.call_tool()
123     async def call_tool(name: str, arguments: Dict[str, Any]) -> List[Dict[str, Any]]:
124         if name == "标注图片":
125             image_data = arguments.get("image_data", [])
126             box_points = arguments.get("box_points", "")
127             api_key = arguments.get("api_key", "")
128             chat_id = arguments.get("chat_id", "")
129             agent_id = arguments.get("agent_id", "")
130             model = arguments.get("model", "Qwen")
131             print(arguments)
132             result = start_tag(image_data, box_points, api_key, chat_id, agent_id, model)
133             print(f"result type:{type(result)}")
134             return [{"type": "text", "text": json.dumps(result,ensure_ascii=False)}]
```

根据工具名称调用对应的入口api

注意,函数返回结果 result 必须为String类型

3. 部署位置 137 /home/workspace/mcp-geosophon

4. geosophon mcp

